`weatherisk`: A Python Package for Spatial Clustering of Climate Extremes via Max-Stable Processes

Widad Kchikach[a,b,*], Thorsten Dickhaus[b], Gerrit Lohmann[a,b]

[a] *Alfred Wegener Institute Helmholtz Centre for Polar and Marine Research, Bremerhaven, Germany*
[b] *University of Bremen, Bremen, Germany*

Abstract

We present `weatherisk`, an open-source Python package that identifies regions of similar extreme-weather behaviour by clustering gridded climate data. Starting from daily observations, the software extracts annual maxima, fits extreme-value distributions, and models the spatial dependence of extremes using max-stable processes. It then estimates local anisotropy parameters at every grid cell and groups cells into clusters where the dependence structure is homogeneous. Two complementary clustering methods are provided: one based on the overlap of locally estimated anisotropy ellipses (LEC) and one based on pairwise extremal dependence coefficients (EDC). The package faithfully reproduces an established R-based pipeline in Python, verified by 50 automated cross-validation tests, and adds a command-line interface, scalable estimation strategies, and real-data pipelines for NOAA CPC climate reanalysis products.

Keywords: spatial extremes, max-stable process, spatial clustering, anisotropy, Python, composite likelihood

Metadata

1. Motivation and significance

Heatwaves and heavy rainfall events rarely affect a single location in isolation. When one city records an extreme temperature, nearby cities usually experience the same event. Understanding *where* extreme weather tends to strike together is important for disaster planning, insurance, and climate

*Corresponding author
Email address: `widad.kchikach@awi.de` (Widad Kchikach)

Nr.	Code metadata description	Metadata
C1	Current code version	v0.1.0
C2	Permanent link to code/repository	`https://github.com/xxx/weatherisk`
C3	Permanent link to Reproducible Capsule	(to be added)
C4	Legal Code License	MIT
C5	Code versioning system used	git
C6	Software code languages, tools, and services used	Python 3
C7	Compilation requirements, operating environments & dependencies	Python ≥3.10; NumPy; SciPy; pandas; matplotlib; Click; xarray; netCDF4; scikit-learn; Cartopy
C8	Link to developer documentation	`https://github.com/xxx/weatherisk`
C9	Support email for questions	widad.kchikach@awi.de

Table 1: Code metadata (mandatory).

adaptation. If this spatial co-occurrence is ignored, the total impact of a large-scale event can be severely underestimated [1].

The statistical theory of *max-stable processes* provides a natural way to describe how extremes at different locations depend on each other [2,3]. In practice, one fits local parameters that characterise how strongly (and in which direction) extreme events at each grid point are linked to their neighbours. Once these parameters are estimated, grid points with similar dependence structure can be grouped into *clusters*—regions where extreme events behave in a coherent way.

An R-based pipeline for this type of spatial clustering was developed by Contzen et al. [4]. It simulates max-stable processes, estimates local anisotropy parameters through pairwise composite likelihood, builds a dissimilarity matrix from ellipse overlaps, and applies hierarchical clustering. However, the original implementation is split across several R scripts orchestrated by SLURM shell files, making it difficult to install, test, or extend.

`weatherisk` solves this problem by re-implementing the full pipeline in a single Python package that can be installed with `pip install`. It faithfully reproduces every step of the original R code—verified by 50 automated tests that compare intermediate results to machine precision—and adds several practical features:

- a command-line interface for running the pipeline with a single com-

mand;

- extreme-value pre-processing (fitting distributions to annual maxima) so that raw climate data can be used directly;
- a real-data pipeline that reads NOAA CPC daily NetCDF files and produces publication-ready cluster maps; and
- a scalable strategy that allows global grids with hundreds of thousands of cells to be processed on standard HPC hardware.

No existing Python package offers this combination of max-stable simulation, local composite-likelihood estimation, and ellipse-based spatial clustering. Packages such as `pyextremes` [5] and `scikit-extremes` focus on univariate extreme-value analysis; the R package `SpatialExtremes` [6] provides max-stable simulation but not the clustering pipeline.

2. Software description

2.1. Software architecture

`weatherisk` is a pip-installable Python package organised into 18 modules and a CLI entry point. Figure 1 shows the overall workflow.

The processing pipeline consists of five stages:

1. **Data preparation.** Raw daily climate data (NetCDF) are loaded, annual block maxima are extracted, and extreme-value distributions (GEV) are fitted at each grid cell. The fitted values are transformed to a common unit (Fréchet margins) so that spatial dependence can be compared across locations.
2. **Simulation** (for validation on synthetic data). Non-stationary max-stable processes are generated on a regular grid with known parameters, following the spectral representation of Schlather [7].
3. **Local estimation.** At each grid cell, the three anisotropy parameters—two semi-axis lengths and a rotation angle—are estimated by maximising a pairwise composite likelihood over neighbouring pairs [3]. Estimates are then spatially smoothed.
4. **Clustering.** A pairwise dissimilarity matrix is computed from the estimated parameters, and hierarchical agglomerative clustering groups the grid cells into regions. Two dissimilarity measures are available: ellipse-overlap (LEC) and madogram-based extremal coefficients (EDC, following Saunders et al. [8]).
5. **Refinement.** Within each cluster, the dependence parameters are re-estimated using all data from the cluster's grid cells, and the goodness-of-fit of each method is assessed.

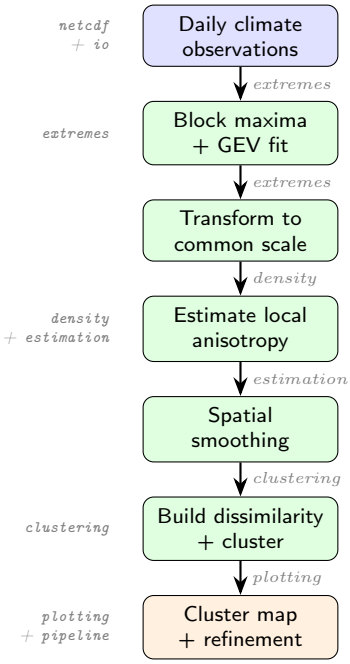

Figure 1: Processing pipeline of `weatherisk`. Boxes show the main stages; labels on the right indicate the module responsible; labels on the left show the corresponding Python module names.

2.2. Software functionalities

Covariance modelling (`covariance`).. This module computes the anisotropic covariance between any two grid cells, given their semi-axis lengths and rotation angle. It supports both the stationary case (constant parameters everywhere) and the non-stationary case (parameters vary in space), as well as the conversion between covariance values and extremal dependence coefficients.

Simulation (`simulation`).. Generates synthetic max-stable realisations with spatially varying anisotropy. This is used to validate the clustering method on data where the true underlying structure is known.

Estimation (`density`, `estimation`).. Estimates the three local parameters at each grid cell by fitting the pairwise dependence model to all nearby pairs. Multiple random starting points are used to avoid poor local optima. Results are spatially smoothed, and circular wrapping is applied to the rotation angle.

Clustering (`clustering`).. Computes the full pairwise dissimilarity matrix and applies average-linkage hierarchical clustering. The number of clusters is determined automatically by a quantile threshold on the dendrogram merge heights. Two dissimilarity measures are implemented:

- **LEC** (Local Estimate Clustering): based on the Jaccard-like overlap of the anisotropy ellipses;
- **EDC** (Extremal Dependence Clustering): based on rank-estimated pairwise extremal coefficients [8,9].

Real-data support (`netcdf`, `extremes`).. Reads NOAA CPC daily precipitation and temperature files, extracts annual block maxima, fits GEV distributions, and transforms to unit Fréchet margins for dependence modelling.

Scalability (`scalable`).. For large grids, local estimation runs at full resolution in parallel, but clustering operates on a coarsened grid that fits in memory. Labels are then propagated back to the fine grid and refined.

Command-line interface (`cli`).. A single entry point exposes the main pipeline flows:

```
$ weatherisk validate --params stripes \
    --resolution 10 --n-sim 20 --seed 42
$ weatherisk maps --variable precip
```

3. Illustrative examples

We demonstrate `weatherisk` with three examples: one on synthetic data and two on real precipitation observations.

3.1. Example 1: Recovering known structure from synthetic data

In this example, we simulate max-stable data on a 51×51 grid where the semi-major axis b increases linearly from left to right, creating vertical "stripe" regions. The pipeline should recover these stripes from the simulated data alone.

```
from weatherisk.parameters import get_preset
from weatherisk.pipeline import run_pipeline

result = run_pipeline(
    resolution=51, n_sim=20,
    preset="stripes", seed=42,
)
```

Listing 1: Running the stripes validation.

Figure 2 shows the result. Panel (a) displays the EDC clusters, panel (b) the LEC clusters, and panel (c) the true parameter field. Both methods successfully recover the stripe boundaries, with LEC producing sharper transitions that closely match the ground truth. This confirms that the Python implementation correctly reproduces the clustering methodology of Contzen et al. [4].

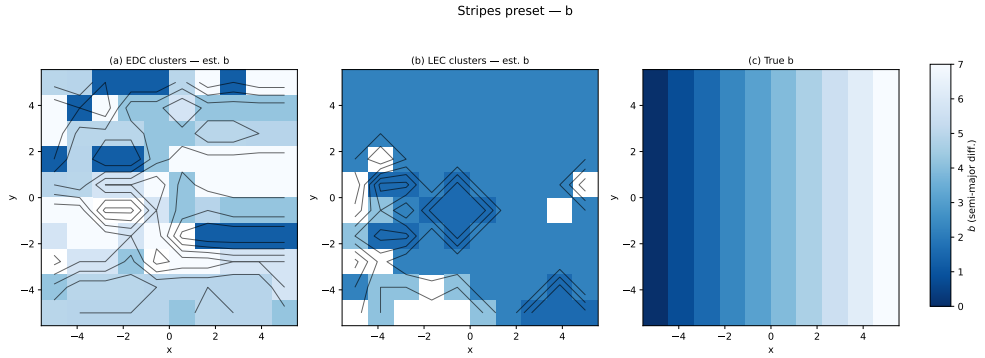

Figure 2: Synthetic validation on the "stripes" preset. (a) EDC clusters coloured by estimated b; (b) LEC clusters coloured by estimated b; (c) true parameter field. Both methods recover the stripe structure; LEC produces boundaries closer to the true transitions.

3.2. Example 2: LEC clustering of precipitation extremes

We apply the full real-data pipeline to 20 years (2000–2019) of NOAA CPC daily precipitation over a European–Middle Eastern domain (30°–65°N, 5°–55°E), coarsened to approximately 2° resolution (384 land cells).

```
$ weatherisk maps --variable precip
```

Listing 2: Running the CPC maps pipeline.

Figure 3 shows the resulting LEC cluster map. The method identifies $k = 26$ clusters based on ellipse-overlap dissimilarity with a 30%-quantile threshold. The spatial pattern is physically interpretable: the Mediterranean basin, Atlantic-influenced Western Europe, Scandinavia, and continental interiors are assigned to different clusters, consistent with known climatic boundaries. The full pipeline—data loading, GEV fitting, local estimation, smoothing, clustering, and map generation—runs in approximately 100 seconds on an Apple M2 laptop with 16 GB RAM.

3.3. Example 3: EDC clustering of precipitation extremes

For comparison, Figure 4 shows the EDC cluster map produced from the same data. The EDC method, which uses madogram-based extremal coefficients rather than locally estimated ellipse parameters, identifies $k = 18$ clusters. Because the madogram captures a different aspect of spatial dependence (the average tendency of two locations to jointly exceed high thresholds), the resulting clusters are coarser and more fragmented than the LEC clusters. Together, the two methods provide complementary views of the dependence structure: LEC captures the shape and orientation of spatial dependence, while EDC captures its overall strength.

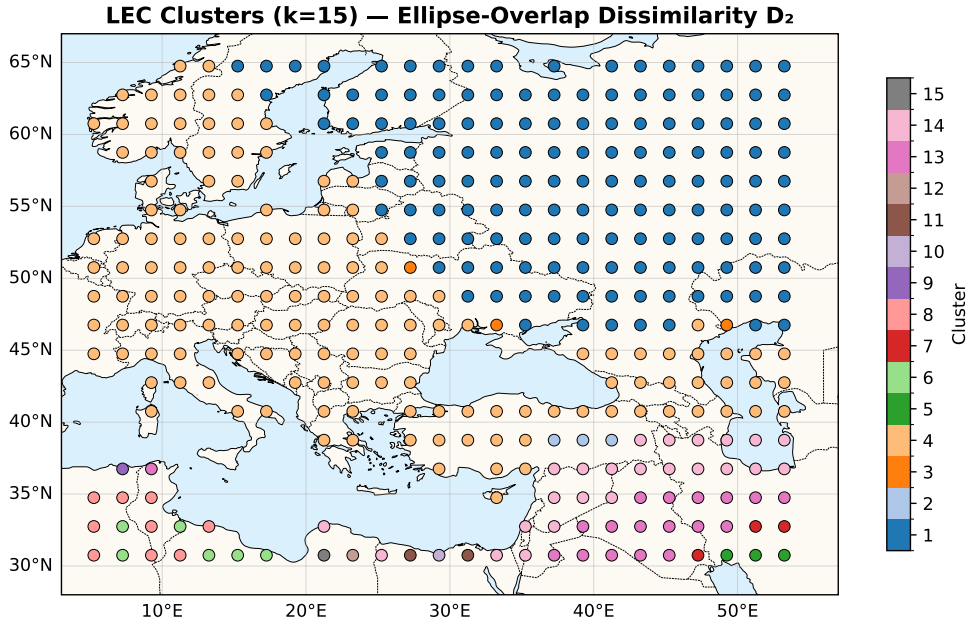

Figure 3: LEC spatial dependence clusters for precipitation extremes (2000–2019, $k = 26$). Each colour identifies a region where extreme precipitation events share a similar spatial dependence structure.

4. Impact

`weatherisk` makes an established spatial-extreme clustering methodology accessible as a standard Python package, removing the barrier of multi-language R/Shell/SLURM workflows.

Reproducibility and correctness.. The package includes 105 automated tests, of which 50 are cross-validation and R-parity tests. These tests execute the original R functions (via exported reference data), compare every intermediate result—from individual covariance evaluations to final cluster assignments—and assert agreement to machine precision. Five specific algorithmic fixes (parameter rescaling, boundary retries, column-major indexing, gamma-wrapping, and average log-likelihood computation) were implemented to achieve exact numerical equivalence with R. This level of verification gives confidence that published results based on the R code can be faithfully reproduced in Python.

New research directions.. By providing a clean Python API and CLI, `weatherisk` lowers the entry barrier for researchers who wish to apply spatial clustering of extremes to new domains (e.g., wind speed, sea-level pressure) or different datasets (e.g., ERA5, CMIP6 projections). The modular architecture also

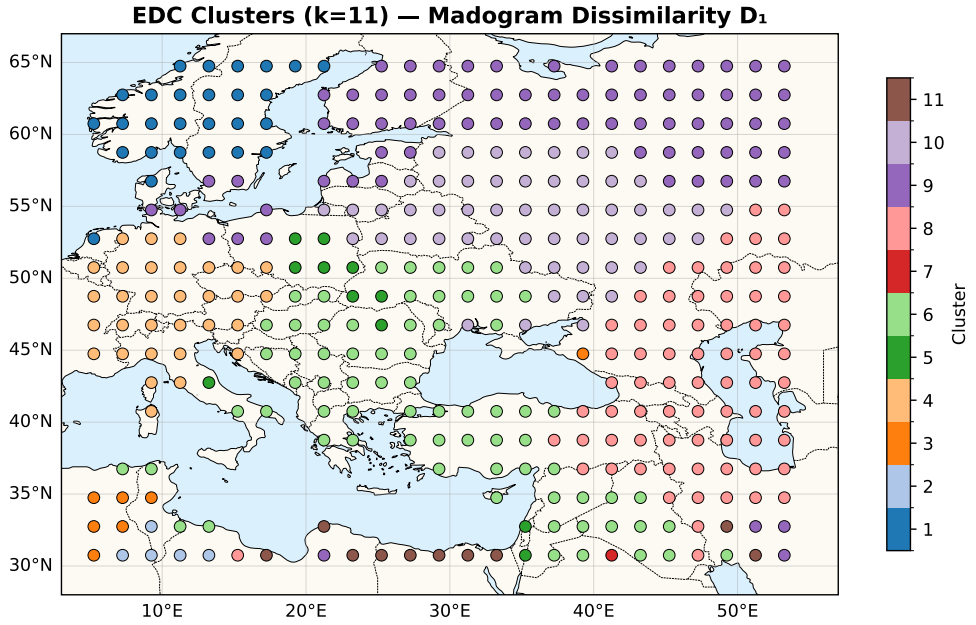

Figure 4: EDC spatial dependence clusters for precipitation extremes (2000–2019, $k = 18$). Compared with the LEC clusters in Figure 3, the EDC clusters are coarser and more fragmented, reflecting a different measure of spatial dependence.

supports extensions such as per-cluster tail-risk metrics (Value-at-Risk, Expected Shortfall) and economic exposure weighting, which are planned for a future release.

Education.. The step-by-step pipeline design, together with the comprehensive test suite, makes the package suitable for graduate-level teaching of spatial extreme-value methods.

Scalability.. The coarse-grid proxy strategy and embarrassingly parallel local estimation allow the method to be applied to global grids with over 250 000 cells on standard university HPC infrastructure.

5. Conclusions

We have presented `weatherisk`, a Python package that implements the complete pipeline for spatial clustering of climate extremes via max-stable processes. The software consolidates a fragmented R/Shell workflow into a single, tested, pip-installable library. Key contributions include:

1. a verified Python port of the entire clustering pipeline, proven equivalent to the original R code by 50 automated tests;

2. extreme-value pre-processing for real climate data (GEV fitting, Fréchet transformation);
3. a real-data pipeline for NOAA CPC daily observations that produces publication-ready cluster maps (Figures 3 and 4); and
4. scalable strategies for global grids.

The package is designed with extensibility in mind: future releases will add per-cluster risk metrics, economic exposure weighting, and support for climate projection scenarios. `weatherisk` is available under the MIT licence.

Acknowledgements

The authors thank [to be completed] for computational resources and helpful discussions.

Appendix A. Key code examples

This appendix collects the main code snippets that a practitioner needs to run the two pipeline flows shown in the paper.

Appendix A.1. A.1 Synthetic validation (Flow 1)

```
from weatherisk.parameters import get_preset
from weatherisk.simulation import simulate_non_stat
from weatherisk.grid import make_grid
from weatherisk.estimation import (
    calc_local_estimates,
    smooth_estimates,
    calc_estimates_in_clusters,
)
from weatherisk.clustering import (
    ellipse_dissimilarity,
    cluster_from_dissimilarity,
    saunders_clustering,
)
from weatherisk.plotting import plot_cluster_map

# 1. Load a parameter preset
p = get_preset("stripes")

# 2. Build a grid and simulate data
grid = make_grid(resolution=10)
data = simulate_non_stat(
    p.params, grid, n_sim=20,
    df=p.df, alpha=p.alpha, seed=42,
)
```

```
# 3. Estimate local anisotropy parameters
raw_est = calc_local_estimates(
    data, grid, df=p.df, alpha=p.alpha,
)
est = smooth_estimates(raw_est, grid)

# 4. Cluster by ellipse-overlap dissimilarity
D = ellipse_dissimilarity(est, grid)
clusters = cluster_from_dissimilarity(
    D, quantile_threshold=0.3,
)

# 5. Re-estimate within clusters
in_clust = calc_estimates_in_clusters(
    data, clusters, grid,
    df=p.df, alpha=p.alpha,
)

# 6. Plot
fig = plot_cluster_map(clusters, grid)
fig.savefig("lec_clusters.pdf")
```

Listing 3: Complete synthetic validation workflow.

Appendix A.2. A.2 Real-data pipeline (Flow 2)

```
# Run the full real-data pipeline from the
# command line (NetCDF files in data/netcdf/):
$ weatherisk maps --variable precip

# Or in Python:
from weatherisk.cpc_pipeline import run_cpc_maps
results = run_cpc_maps(variable="precip")
```

Listing 4: CPC precipitation clustering via CLI.

Appendix A.3. A.3 Comparing LEC and EDC methods

```
from weatherisk.pipeline import run_pipeline
from weatherisk.plotting import (
    plot_cluster_map,
    plot_eici_comparison,
)

# Run full pipeline (includes both methods)
result = run_pipeline(
    resolution=10, n_sim=20,
    preset="stripes", seed=42,
```

```
)

# Access cluster labels
lec_labels = result["lec_clusters"]
edc_labels = result["edc_clusters"]

# Compare goodness-of-fit
fig = plot_eici_comparison(result)
fig.savefig("eici_comparison.pdf")
```

Listing 5: Running both clustering methods and comparing results.

References

[1] A.C. Davison, S.A. Padoan, M. Ribatet, Statistical modeling of spatial extremes, Stat. Sci. 27 (2) (2012) 161–186.

[2] L. de Haan, A. Ferreira, Extreme Value Theory: An Introduction, Springer, New York, 2006.

[3] S.A. Padoan, M. Ribatet, S.A. Sisson, Likelihood-based inference for max-stable processes, J. Am. Stat. Assoc. 105 (489) (2010) 263–277.

[4] K. Contzen, T. Dickhaus, Clustering by local extremal properties, Working paper, 2024.

[5] G. Bocharov, pyextremes: Extreme value analysis in Python, `https://github.com/georgebv/pyextremes`, 2023.

[6] M. Ribatet, SpatialExtremes: Modelling Spatial Extremes, R package, `https://CRAN.R-project.org/package=SpatialExtremes`, 2023.

[7] M. Schlather, Models for stationary max-stable random fields, Extremes 5 (1) (2002) 33–44.

[8] K. Saunders, A.G. Stephenson, P.J. Taylor, D. Karoly, The spatial clustering of rainfall extremes and the influence of El Niño–Southern Oscillation, J. Climate 34 (12) (2021) 4859–4878.

[9] D. Cooley, P. Naveau, P. Poncet, Variograms for spatial max-stable random fields, in: P. Bertail, P. Soulier, P. Doukhan (Eds.), Dependence in Probability and Statistics, Springer, 2006, pp. 373–390.